SISTEMA INTEGRADO DE GESTÃO • GRUPO BAMBOO

Consolidação dos 5 Sistemas Legados + Otimizador Inteligente de Carga 3D + IA

Engenharia & Arquitetura: Engenheiro de Software Sênior & Arquiteto | **Data:** Setembro/2026 | **Versão:** 1.0 Final

1. Visão Geral e Princípio Arquitetural Central

A plataforma unifica os 5 projetos isolados da Hostinger em um **Monolito Modular com Clean Architecture**. O princípio cardeal do sistema é a **extinção de cadastros duplicados** por meio de um Cadastro Mestre único (Produto, Fornecedor, Cliente, Depósito, Veículo, Usuário, Empresa e Centro de Custo), garantindo que todos os módulos de negócio consumam a mesma fonte de verdade sem redundâncias ou divergências.

Camada	Responsabilidade no Monolito Modular
Domain	Entidades de negócio puras (dataclasses), Value Objects, invariantes e interfaces abstratas de repositório. Zero dependência de frameworks ou ORM.
Application	Casos de Uso (Use Cases), DTOs e Facades Públicas de integração intermódulos. Orquestra a execução das regras de negócio.
Infrastructure	Implementações de persistência com SQLAlchemy 2.0 Async, Alembic, adaptadores matemáticos (py3dbp) e clientes de IA (LLM).
Presentation	Roteadores FastAPI, validação estrita com Pydantic v2 e injeção de dependências de autenticação e RBAC.

Regra de Ouro Intermódulos: Módulos não acessam o banco de dados uns dos outros diretamente. Toda comunicação é realizada exclusivamente via interfaces públicas (Facades / DTOs) da camada Application ou por eventos de domínio transacionais.

2. Racional Arquitetural: FastAPI + SQLAlchemy 2.0 vs. Django

Embora o Django ofereça o *Django Admin* pronto de fábrica, a escolha técnica do **FastAPI com SQLAlchemy 2.0** é superior e mais robusta para as características centrais deste sistema:

- **Data Mapper vs. Active Record:** O Django ORM acopla fortemente a entidade à tabela do banco. O SQLAlchemy 2.0 permite que as entidades de Domínio sejam classes puras, viabilizando o TDD sem depender de banco.
- **Concorrência Assíncrona & IA:** O FastAPI é nativo ASGI/asyncio, indispensável para streaming de respostas de IA (LLM Tool Calling) e cálculos de empacotamento 3D sem bloquear a API.
- **Tipagem de Ponta a Ponta:** Modelos Pydantic v2 geram OpenAPI automático, permitindo sincronização estrita de tipos com o frontend React e TypeScript.

3. Módulos Consolidados do Sistema

Módulo	Escopo e Integrações Chave
Cadastros Mestres	Cadastro único de Produtos (dimensões X/Y/Z, peso, empilhamento, fragilidade), Fornecedores, Clientes, Depósitos, Empresas, Centros de Custo e Veículos.
Compras Nacionais	PO sequencial, itens, recebimentos parciais, integração com entrada de estoque e contas a pagar.
Importações	Processos internacionais, câmbio, BL, container, portos, LI, INMETRO, cálculo de Landed Cost e rateio.
Estoque	Saldos por depósito, movimentações rastreáveis (origem, destino, usuário, motivo), inventário e reservas.
Comercial / Vendas	Pedidos de venda, representantes, comissões, metas, reserva automática de estoque e geração de títulos a receber.
Produção & Expedição	Apontamento de produção, ordens de separação, romaneio, comprovantes de entrega e vínculo com plano de carga.
Financeiro	Contas a pagar/receber, baixas parciais/totais, conciliação bancária, fluxo de caixa, empréstimos e DRE.
Recursos Humanos	Cadastro de funcionários, evolução salarial, folhas de pagamento, férias e integração com contas a pagar.
Otimizador de Carga 3D	Motor matemático determinístico de 3D Bin Packing: cálculo de cubagem, peso máximo, empilhamento e eixos.
AI Agent (Tool Calling)	Assistente operacional via LLM: interpretação de linguagem natural, acionamento do motor 3D e explicação de planos.
Auditoria & Governança	Log imutável de ações críticas (quem, quando, o quê, valores anteriores e posteriores) e notificações.

4. Diretrizes Obrigatórias: TDD, Clean Code e Segurança

- **Ciclo TDD Rígido:** Nenhuma regra de negócio é codificada antes da escrita do teste que falha (*Red*). Em seguida cria-se o código que satisfaz o teste (*Green*) e realiza-se a refatoração contínua (*Refactor*).
- **Clean Code:** Responsabilidade Única (SRP), Inversão de Dependência (DIP), zero variáveis mágicas, funções curtas e type hints em 100% da base.
- **Segurança OWASP API Top 10:** Hashing seguro (Argon2 / Bcrypt), autenticação JWT com expiração rígida, RBAC validado por testes de permissão no backend (Admin, Compras, Importação, Estoque, Financeiro, Comercial, RH, Gestor), sanitização e imunidade a SQL Injection.

5. Roadmap de Implementação e Condições de Conclusão (DoD)

Fase 1: Infraestrutura de Containers e Esqueleto Modular Clean Arch

Task 1.1 — Orquestração Docker e Estrutura de Diretórios

Configuração do docker-compose.yml (PostgreSQL 16, Backend FastAPI, Frontend React), Dockerfiles otimizados, .env.example e criação da árvore de pastas nas 4 camadas dos módulos.

✓ Condição de Conclusão: docker compose config validado sem erros; árvore de diretórios criada e documentada; ambiente pronto para build.

Fase 2: Core, Segurança, Autenticação RBAC e Banco Assíncrono (TDD)

Task 2.1 — Conexão Assíncrona e Alembic

Setup do SQLAlchemy 2.0 Async, pool de conexões e Alembic para migrações assíncronas.

✓ Condição de Conclusão: Teste automatizado de ciclo de vida do banco e migração inicial executados com sucesso.

Task 2.2 — Domínio de Usuários e Segurança com TDD

Entidades User, Role (Admin, Compras, Importação, Estoque, Financeiro, Comercial, RH, Gestor), Value Objects Email e PasswordHash.

✓ **Condição de Conclusão: 100% de cobertura nos testes unitários das entidades de autenticação e validação de senhas/roles.**

Task 2.3 — Casos de Uso de Auth e Repositórios

Casos de uso RegisterUserUseCase, AuthenticateUserUseCase, geração e validação de JWT assinado com segredo forte.

✓ **Condição de Conclusão: Testes unitários com repositórios em memória e testes de integração com banco PostgreSQL aprovados.**

Task 2.4 — Presentation e Bateria de Testes de Segurança

Rotas /api/v1/auth/login, /register, /me e dependência RBAC. Suíte de testes contra escalada de privilégio, tokens inválidos/expirados e auditoria estática do Bandit.

✓ **Condição de Conclusão: Suíte de testes de segurança 100% verde; zero alertas de vulnerabilidade alta/crítica no Bandit.**

Fase 3: Cadastros Mestres, Compras e Estoque Básico (MVP 1)

Task 3.1 — Domínio e Repositórios de Cadastros Mestres

Entidades: Product (comprimento, largura, altura, peso, empilhamento, fragilidade, eixos de rotação), Supplier, Customer, Warehouse, Vehicle.

✓ **Condição de Conclusão: Testes unitários cobrindo validação dimensional de produtos e regras de unicidade de cadastros mestres.**

Task 3.2 — Módulo de Estoque e Rastreabilidade

Entidades e casos de uso de movimentação (cabeçalho + itens), saldos por depósito, inventário de contagem e motivos padronizados.

✓ **Condição de Conclusão: Impossibilidade matemática de alteração direta de saldo; toda movimentação registrada com rastreabilidade auditável.**

Task 3.3 — Módulo de Compras Nacionais Integrado ao Estoque

Pedidos de compra (PO sequencial), status, recebimentos parciais e geração de entrada no estoque via interface pública.

✓ **Condição de Conclusão: Recebimento de compra gera entrada de saldo no estoque e histórico rastreável via testes de integração.**

Fase 4: Comercial, Expedição e Financeiro Integrado (MVP 2)

Task 4.1 — Vendas, Representantes e Reserva de Estoque

Pedidos de venda, comissões, metas e disparo de evento transacional para reserva de estoque na confirmação do pedido.

✓ **Condição de Conclusão: Testes de concorrência garantindo que pedidos simultâneos não provoquem saldo negativo.**

Task 4.2 — Produção, Expedição e Romaneios

Ordens de separação, baixa definitiva do estoque na expedição e geração de romaneios vinculados a veículos.

✓ **Condição de Conclusão: Expedição baixa estoque reservado e gera documento de entrega com integridade transacional.**

Task 4.3 — Financeiro Integrado (AP/AR, Caixa e Bancos)

Contas a pagar geradas de Compras e contas a receber geradas de Vendas; baixas parciais/totais e fluxo de caixa.

✓ **Condição de Conclusão: Testes de integração comprovando liquidação financeira e consistência em operações parciais.**

Fase 5: Módulo de Importações e Landed Cost (MVP 3)

Task 5.1 — Processo de Importação, Rastreio e Documentos

Pedidos internacionais, BL, container, portos, cronograma ETA, licenças de importação (LI) e conformidade INMETRO.

✓ **Condição de Conclusão: Ciclo de vida da importação e checklists de tarefas operando com integridade documental.**

Task 5.2 — Memória de Cálculo do Landed Cost e Câmbio

Cálculo do custo final da mercadoria com rateio de fretes, impostos, taxas, armazenagem e variação cambial.

✓ **Condição de Conclusão: Testes matemáticos validando 100% do rateio de custos e atualização do custo final do produto.**

Fase 6: Otimizador Inteligente de Carga 3D & AI Agent (MVP 4)

Task 6.1 — Motor Determinístico de 3D Bin Packing (py3dbp)

Algoritmo de empacotamento espacial considerando dimensões úteis do baú, peso máximo, eixos de rotação, empilhamento e caixas frágeis.

✓ Condição de Conclusão: Testes unitários com lotes reais comprovando que nenhuma caixa ultrapassa o baú ou sobrecarrega o peso permitido.

Task 6.2 — Application Facade e Schemas de Tool Calling

Criação da LogisticsOptimizationFacade e especificação estrita dos schemas Pydantic das ferramentas do agente de IA.

✓ Condição de Conclusão: Schemas de tools validados e executáveis pelo dispatcher do agente de IA.

Task 6.3 — AI Agent, Chat e Visualização 3D

Endpoint /api/v1/ai/chat respondendo em linguagem natural com base nos cálculos do motor e renderização 3D no React.

✓ Condição de Conclusão: Prompt 'Quantas caixas padrão cabem no baú X?' aciona o motor via Tool Calling e retorna coordenadas 3D e resposta estruturada.

Fase 7: Recursos Humanos, DRE Avançado e Homologação Final

Task 7.1 — Módulo de RH e Folha Integrada

Funcionários, evolução salarial, folhas de pagamento com envio automático para Contas a Pagar do Financeiro.

✓ Condição de Conclusão: Folha aprovada gera lançamento no fluxo financeiro com rastreabilidade.

Task 7.2 — Auditoria Global, SAST e Subida Idempotente

Trilha de auditoria imutável, relatório Bandit limpo, cobertura de testes > 85% e execução total via docker compose up --build.

✓ Condição de Conclusão: Sistema completo rodando unificado em containers com migrações automáticas e healthchecks verdes.

6. Estrutura de Diretórios Inicial Planejada

7. Próximo Passo Imediato

Aprovação do documento e início imediato da **Etapa 1** (criação do docker-compose.yml, containers e estrutura inicial de diretórios do Backend).